

Day 117: Scikit-learn Pipelines and a Beginner Kaggle Workflow

1. Day number

Day 117

2. Topic name

Practical End-to-End Machine Learning Workflow

Today you will connect several things you learned separately:

```
data
→ preprocessing
→ train/test split
→ model training
→ prediction
→ evaluation
```

The main new idea is a **scikit-learn Pipeline**, which helps keep preprocessing and modeling steps together.

3. Connection

Earlier, you learned about:

- cleaning data,
- encoding categorical columns,
- scaling numerical features,
- train/test splits,
- supervised-learning models,
- evaluation metrics.

Until now, these may have felt like separate topics.

Today you will combine them into one small repeatable workflow:

```
raw dataset
  ↓
inspect
  ↓
prepare features and target
  ↓
split
  ↓
preprocess
  ↓
train model
  ↓
```

```
predict
  ↓
evaluate
```

This is close to the basic workflow you would use for a small real-world machine-learning project.

4. Important topics

Scikit-learn estimator

In scikit-learn, an **estimator** is generally an object that learns something from data.

For example:

```
LogisticRegression()
DecisionTreeClassifier()
LinearRegression()
```

These models commonly provide methods such as:

```
fit()
predict()
```

Other scikit-learn objects, such as preprocessing transformers, may also follow a similar interface.

Preprocessing

Preprocessing means preparing raw data before giving it to the model.

Examples include:

```
missing values
→ fill or handle them

text categories
→ convert to numerical form

numerical features
→ possibly scale them
```

For example:

	Age	City
	25	Delhi
	40	Mumbai

A model may not directly understand:

```
Delhi  
Mumbai
```

so the city column may need encoding.

fit()

fit() means:

Learn from the provided data.

For a model:

```
model.fit(X_train, y_train)
```

means the model learns the relationship between the training features and target.

For a preprocessing step, fit() may instead learn information such as:

- category names,
 - averages used for missing-value filling,
 - scaling statistics.
-

predict()

After training, we can use:

```
model.predict(X_test)
```

to generate predictions for unseen examples.

Conceptually:

```
training data  
  ↓  
fit()  
  ↓  
trained model  
  ↓  
new data  
  ↓  
predict()  
  ↓  
predictions
```

Pipeline concept

A **Pipeline** combines several processing steps into one workflow.

For example:

```
Fill missing values
    ↓
Scale numerical values
    ↓
Train Logistic Regression
```

Instead of manually running each step separately every time, the pipeline can manage the sequence.

Kaggle dataset

Kaggle provides many datasets that can be downloaded and explored for learning.

For today's lesson, think of Kaggle simply as:

A place where you can obtain small real datasets for practicing machine learning.

You do not need to think about competitions or leaderboards.

Train/test workflow

A basic supervised-learning workflow is:

```
dataset
    ↓
features X + target y
    ↓
train/test split
    ↓
train using training data
    ↓
predict using test features
    ↓
compare predictions with test answers
```

5. Foundational notes

A machine-learning model usually cannot safely operate on messy raw data without preparation.

Suppose your dataset contains:

Age	City	Income	Bought
25	Delhi	30000	No
35	Mumbai	missing	Yes
42	Delhi	70000	Yes

Several issues appear.

City is categorical:

```
Delhi
Mumbai
```

Income contains a missing value.

Before training, you may need:

```
City
→ categorical encoding

Income
→ missing-value handling
```

Then a model can work with the prepared values.

The important idea is:

Preprocessing is part of the machine-learning workflow, not something separate from the model.

6. Why pipelines help

Without a pipeline, you might manually do this:

1. Fill missing values
2. Encode categories
3. Scale values
4. Train model
5. Repeat the same transformations on test data

This can become error-prone.

For example, you might accidentally:

```
fit preprocessing on the entire dataset
```

instead of only using training information.

That can create **data leakage**.

A pipeline helps organize the steps:

```
Pipeline
├── preprocessing
└── model
```

Then you can conceptually use:

```
pipeline.fit(X_train, y_train)
```

The pipeline:

```
fits preprocessing on training data
→ transforms training data
→ trains the model
```

Later:

```
pipeline.predict(X_test)
```

will:

```
apply the learned preprocessing
→ pass the transformed data to the model
→ return predictions
```

This makes the workflow more consistent.

Pipeline mental model

Imagine an assembly line:

```
Raw customer row
    ↓
Missing-value handler
    ↓
Category encoder
    ↓
Optional scaler
    ↓
ML model
    ↓
Prediction
```

The same sequence can be reused whenever new data arrives.

7. Beginner Kaggle workflow

A simple Kaggle-style learning workflow can be:

Step 1: Download a small dataset

Choose a beginner-friendly tabular dataset.

Examples could involve:

```
customers  
houses  
students  
passengers  
products
```

Keep the first project small.

Step 2: Load and inspect it

Use Pandas to inspect:

```
head()  
shape  
columns  
dtypes  
missing values
```

Questions to ask:

```
How many rows exist?  
What columns exist?  
Which column is the target?  
Are there missing values?  
Which columns are numerical?  
Which are categorical?
```

Step 3: Perform simple cleaning

Handle obvious problems such as:

```
missing values  
incorrect types  
unnecessary duplicate rows
```

Do not make cleaning unnecessarily complicated.

Step 4: Separate features and target

Suppose the target is:

Purchased

Then conceptually:

$X = \text{Age, Income, City}$

$y = \text{Purchased}$

Step 5: Split into train and test data

Conceptually:

80% → training

20% → testing

The training portion is used to learn.

The test portion is kept aside for evaluation.

Step 6: Build preprocessing

You may have:

Numerical:

Age

Income

Categorical:

City

A sensible plan could be:

Numerical columns

→ fill missing values

→ optionally scale

Categorical columns

→ fill missing values

→ encode

Step 7: Choose one simple model

For a small classification problem, perhaps:

```
Logistic Regression
```

For a regression problem:

```
Linear Regression
```

For today, use only one simple model.

Step 8: Create a pipeline

Conceptually:

```
preprocessor  
  ↓  
simple model
```

Step 9: Train

```
pipeline.fit(X_train, y_train)
```

Step 10: Predict

```
predictions = pipeline.predict(X_test)
```

Step 11: Evaluate

For classification, you might use:

```
accuracy  
precision  
recall  
F1
```

depending on the problem.

For regression:

```
MAE  
MSE  
RMSE
```

For today's exercise, keep the evaluation simple.

8. Easy example

Suppose you have a tiny customer dataset:

Age	City	Monthly Spending	Purchased
22	Delhi	1500	No
35	Mumbai	4200	Yes
28	Delhi	2300	No
45	Bengaluru	6000	Yes
38	Mumbai	5100	Yes

Your target is:

Purchased

Features:

Age
City
Monthly Spending

The problem is a **classification problem** because the target has categories:

Yes
No

You could design this workflow:

Age + Monthly Spending
→ numerical preprocessing

City
→ categorical encoding

all transformed features
→ Logistic Regression

model
→ predict Purchased

That entire preprocessing-plus-model flow can be represented as one pipeline.

9. Problem statement

Choose a small beginner-friendly **tabular classification dataset**.

For example, imagine a customer dataset containing:

Age	Income	City	Purchased
21	25000	Delhi	No
34	50000	Mumbai	Yes
29	missing	Delhi	No
48	85000	Bengaluru	Yes
37	62000	Mumbai	Yes

Your task is to design a tiny scikit-learn workflow that:

1. Uses Purchased as the target.
2. Uses Age, Income, and City as features.
3. Handles the missing Income.
4. Encodes City.
5. Splits the data into training and testing portions.
6. Uses one simple classification model.
7. Combines preprocessing and the model in a Pipeline.
8. Trains using `fit()`.
9. Makes predictions using `predict()`.
10. Evaluates the predictions with one simple classification metric.

Do not write the full Python solution yet.

10. Concepts used

This exercise combines many earlier topics:

```
Pandas
→ dataset inspection

features and target
→ X and y

missing-value handling
→ preprocessing

categorical encoding
→ preprocessing

train/test split
→ evaluation setup

Logistic Regression
→ classification model
```

Pipeline
→ connect preprocessing + model

fit()
→ learn

predict()
→ generate predictions

accuracy or another metric
→ evaluate

The overall picture is:

Dataset
↓
X and y
↓
Train/Test Split
↓
Pipeline
↓
Predictions
↓
Evaluation

preprocessing
↓
model

11. Thought process

When you receive a new beginner tabular dataset, use this order.

Step 1: Understand the target

Ask:

What am I trying to predict?

If the target is:

Yes / No

you probably have a classification problem.

If it is:

```
House Price
```

you probably have a regression problem.

Step 2: Identify feature types

Separate:

```
numerical features  
categorical features
```

For example:

```
Age → numerical  
Income → numerical  
City → categorical
```

This matters because different column types usually require different preprocessing.

Step 3: Inspect missing values

Ask:

```
Which columns contain missing values?
```

Plan simple handling.

For example:

```
missing numerical value  
→ replace using a suitable summary value
```

or:

```
missing category  
→ use a simple placeholder/category
```

Step 4: Split before learning preprocessing information

Keep the test data separate.

Conceptually:

```
original data  
↓  
train/test split
```

↓
fit preprocessing using training data

This helps prevent test information from influencing training.

Step 5: Choose simple preprocessing

Do only what the model needs.

For example:

```
numerical
→ missing-value handling

categorical
→ missing-value handling
→ encoding
```

You do not need dozens of preprocessing steps.

Step 6: Choose one simple model

For the example:

```
Logistic Regression
```

is enough.

Step 7: Put everything together

Build:

```
preprocessor
    +
model
    ↓
Pipeline
```

Step 8: Train

Conceptually:

```
pipeline.fit(X_train, y_train)
```

Step 9: Predict

```
pipeline.predict(X_test)
```

Step 10: Evaluate

Compare:

```
predicted values  
vs  
actual test values
```

and calculate an appropriate metric.

12. Beginner-friendly pseudocode

```
START  
  
load dataset  
  
inspect:  
    first rows  
    columns  
    data types  
    missing values  
  
choose target column  
  
separate:  
    X = features  
    y = target  
  
split X and y into:  
    training data  
    testing data  
  
identify:  
    numerical columns  
    categorical columns  
  
create numerical preprocessing:  
    handle missing numerical values  
    optionally scale  
  
create categorical preprocessing:  
    handle missing categories  
    encode categories  
  
combine preprocessing steps  
  
choose simple model
```

```
create pipeline:
  preprocessing
  then model

fit pipeline using training data

predict using test data

evaluate predictions

END
```

13. Suggested solving approach: scikit-learn Pipeline

At a conceptual level, scikit-learn might involve components such as:

```
train_test_split

SimpleImputer

OneHotEncoder

StandardScaler

ColumnTransformer

Pipeline

LogisticRegression
```

You do not need to memorize all of them at once.

Think of their roles:

```
SimpleImputer
→ handle missing values

OneHotEncoder
→ convert categories into numerical columns

StandardScaler
→ scale numerical features when useful

ColumnTransformer
→ apply different preprocessing to different columns

Pipeline
→ connect preprocessing to model
```

LogisticRegression
→ make classification predictions

The structure could look conceptually like:

```
numeric columns
  ↓
numeric preprocessing
    \
     \
      → combined preprocessing → model
     /
    /
categorical columns
  ↓
categorical preprocessing
```

And finally:

```
Pipeline(
    preprocessing,
    model
)
```

Do not worry about exact Python syntax yet.

14. Easy edge cases

Edge case 1: Missing values

Suppose:

Age	Income
30	40000
42	missing
28	35000

Some models cannot directly handle the missing value.

So you might include:

```
missing-value handling
```

inside the preprocessing pipeline.

That means training and future prediction use the same rule.

Edge case 2: Categorical column

Suppose:

```
City =  
Delhi  
Mumbai  
Bengaluru
```

A model such as Logistic Regression generally needs numerical inputs.

So:

```
City  
↓  
categorical encoder  
↓  
numeric representation
```

belongs inside your preprocessing.

Edge case 3: New category in test data

Imagine training data contains:

```
Delhi  
Mumbai
```

but the test data contains:

```
Chennai
```

A categorical encoder needs to be configured sensibly so an unseen category does not unexpectedly break the workflow.

You do not need the exact configuration today—just remember that unseen categories can occur.

Edge case 4: Very small dataset

If your dataset contains only:

```
6 rows
```

an 80/20 train/test split may leave only one or two rows for testing.

Your evaluation then becomes unstable and not very informative.

A tiny dataset is fine for learning the workflow, but not for drawing strong conclusions about model quality.

Edge case 5: Accidentally preprocessing before the split

A common mistake is:

```
entire dataset
→ fit preprocessing
→ split afterward
```

This can allow information from the future test set to influence preprocessing.

A safer mental model is:

```
split first
  ↓
fit pipeline on training data
  ↓
apply learned transformations to test data
```

15. Hint only

For the problem in Section 9, start with:

```
Target:
Purchased
```

Then classify the feature columns:

```
Age
→ numerical

Income
→ numerical

City
→ categorical
```

Now ask:

```
What does Income need?
→ missing-value handling

What does City need?
→ encoding
```

Then connect those preprocessing steps to one simple classifier.

Your final mental structure should look approximately like:

```
Raw X
↓
Column preprocessing
↓
Prepared numerical features
↓
Logistic Regression
↓
Prediction
```

And remember the full Day 117 workflow:

```
Download dataset
↓
Inspect
↓
Clean
↓
Choose X and y
↓
Train/Test Split
↓
Pipeline
┌───────────┐
│ preprocessing │
│   ↓         │
│   model      │
└───────────┘
↓
fit()
↓
predict()
↓
evaluate
```

The main lesson is not complicated code. It is learning to build a **small, repeatable workflow where preprocessing and the model stay together**.